Convolution Neural Network

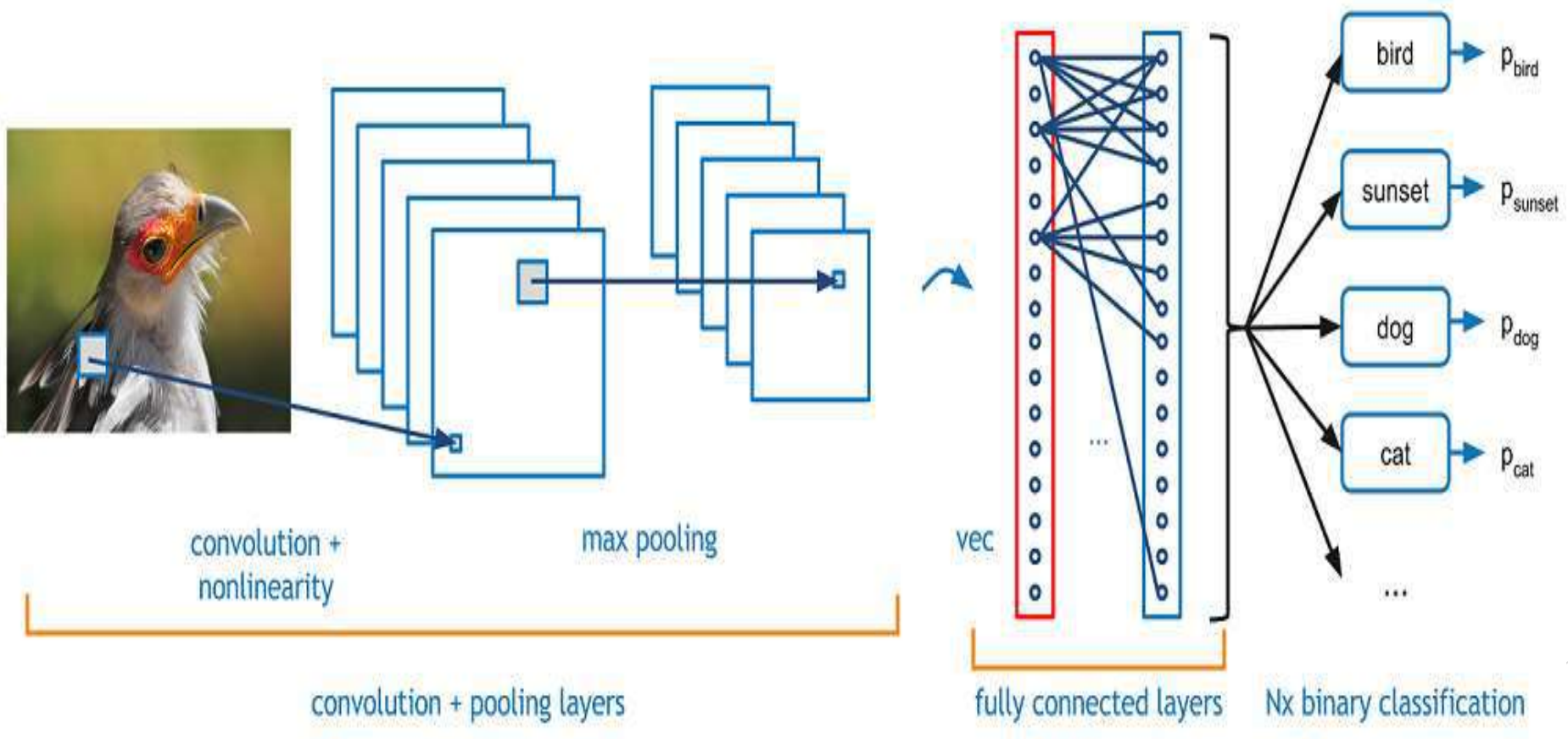
N.Omkar

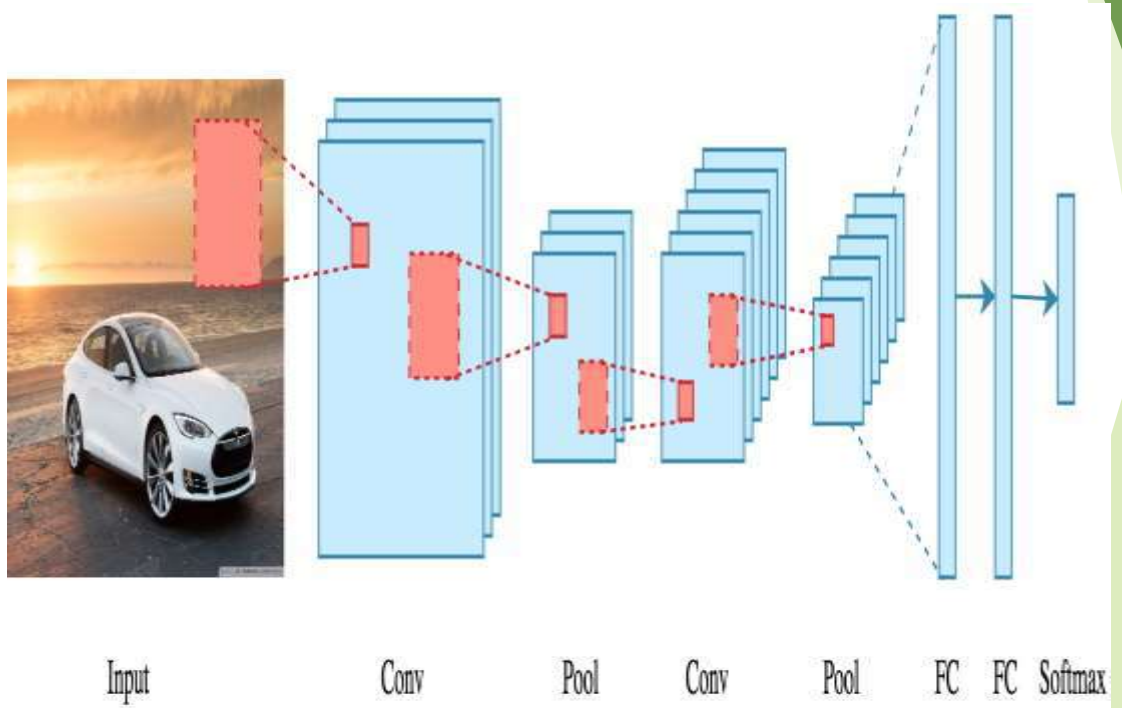
Why CNN

- One of the challenges of computer vision problem that images can be so large and we want a fast and accurate algorithm to work with that.
- For example, a 1000x1000 image will represent 3 million feature/input to the full connected neural network.
- If the following hidden layer contains 1000, then we will want to learn weights of the shape [1000, 3 million] which is 3 billion parameter only in the first layer and thats so computationally expensive!
- One of the solutions is to build this using convolution layers instead of the fully connected layers.

Introduction

Convolutional Neural Networks (CNNs) are neural networks that automatically extract useful features (without manual hand-tuning) from data-points like images to solve some given task like image classification or object detection.





Architecture:

There is an input image that we're working with. We perform a series convolution + pooling operations, followed by a number of fully connected layers. If we are performing multiclass classification the output is softmax. We will now dive into each component.

Convolution:

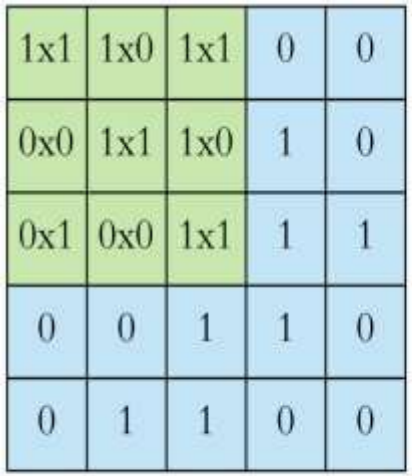
The main building block of CNN is the convolutional layer. Convolution is a mathematical operation to merge two sets of information. In our case the convolution is applied on the input data using a *convolution filter* to produce a *feature map*. There are a lot of terms being used so let's visualize them one by one.

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

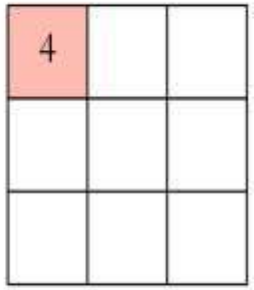
Input

1	0	1
0	1	0
1	0	1

Filter / Kernel



Input x Filter



Feature Map

- On the left side is the input to the convolution layer, for example the input image. On the right is the convolution *filter*, also called the *kernel*, we will use these terms interchangeably. This is called a *3x3 convolution* due to the shape of the filter.
- We perform the convolution operation by sliding this filter over the input. At every location, we do element-wise matrix multiplication and sum the result. This sum goes into the feature map. The green area where the convolution operation takes place is called the *receptive field*. Due to the size of the filter the receptive field is also 3x3.

1	1x1	1x0	0x1	0
0	1x0	1x1	1x0	0
0	0x1	1x0	1x1	1
0	0	1	1	0
0	1	1	0	0

Input x Filter

4	3	

Feature Map

Here the filter is at the top left, the output of the convolution operation “4” is shown in the resulting feature map. We then slide the filter to the right and perform the same operation, adding that result to the feature map as well.

1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

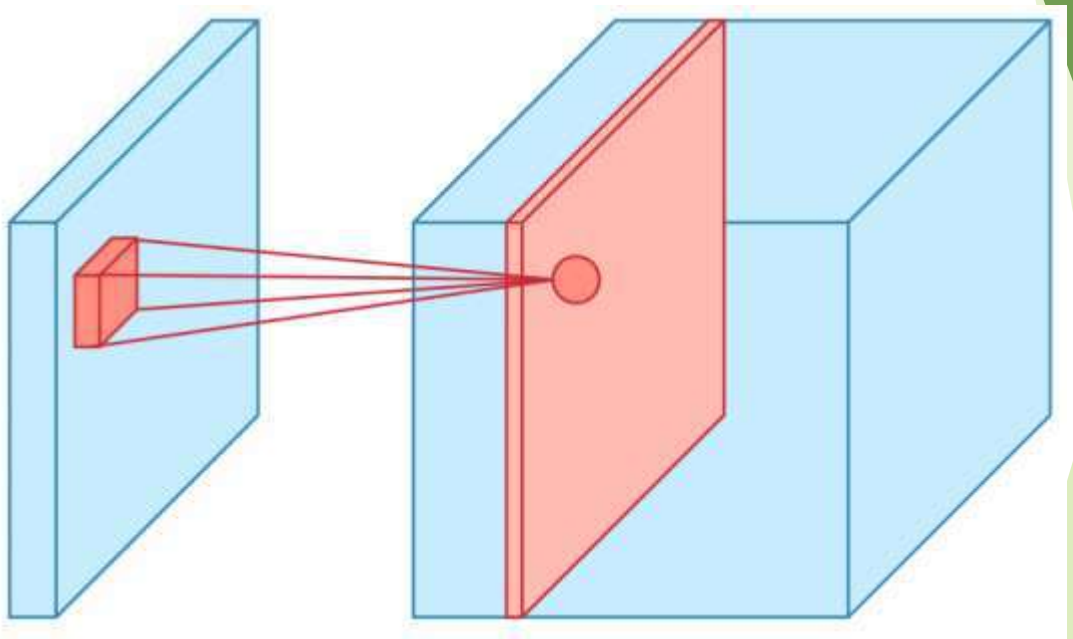
4		

We continue like this and aggregate the convolution results in the feature map. Here's an animation that shows the entire convolution operation.

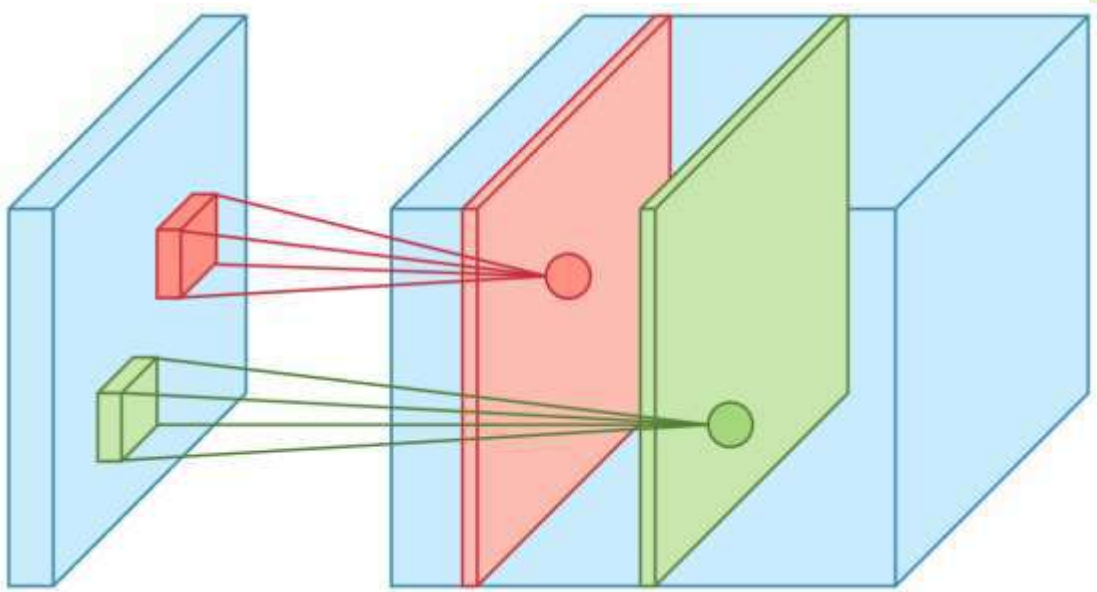
Important Note:

This was an example convolution operation shown in 2D using a 3x3 filter. But in reality these convolutions are performed in 3D. In reality an image is represented as a 3D matrix with dimensions of height, width and depth, where depth corresponds to color channels (RGB). A convolution filter has a specific height and width, like 3x3 or 5x5, and by design it covers the entire depth of its input so it needs to be 3D as well.

One more important point before we visualize the actual convolution operation. We perform multiple convolutions on an input, each using a different filter and resulting in a distinct feature map. We then stack all these feature maps together and that becomes the final output of the convolution layer. But first let's start simple and visualize a convolution using a single filter.



To help with visualization, we slide the filter over the input as follows. At each location we get a scalar and we collect them in the feature map. The animation shows the sliding operation at 4 locations, but in reality it’s performed over the entire input.



The convolution operation for each filter is performed independently and the resulting feature maps are disjoint.

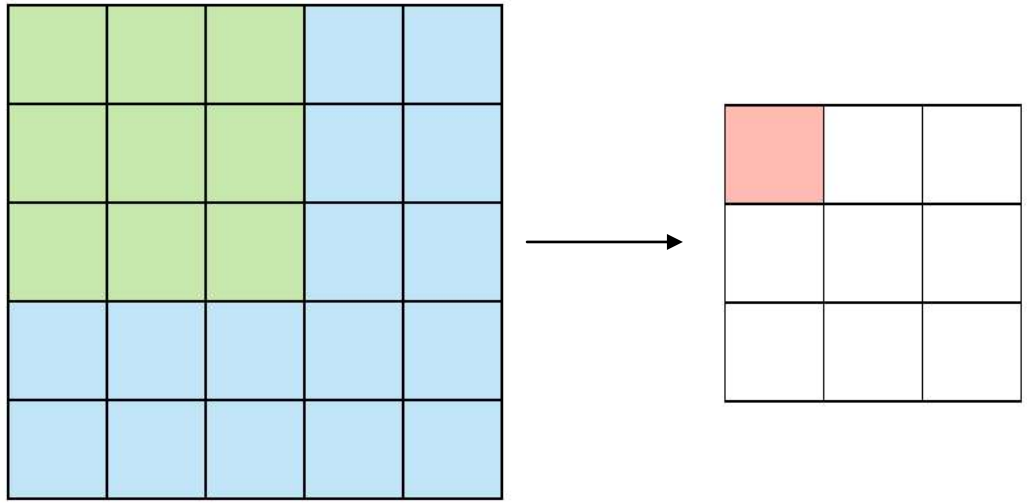
Two color is result of two filter shown in figure

Step-2 Non-linearity

For any kind of neural network to be powerful, it needs to contain non-linearity. The ANN we saw before achieved this by passing the weighted sum of its inputs through an activation function, and CNN is no different. We again pass the result of the convolution operation through *relu* activation function. So the values in the final feature maps are not actually the sums, but the relu function applied to them. We have omitted this in the figures above for simplicity. But keep in mind that any type of convolution involves a relu operation, without that the network won't achieve its true potential.

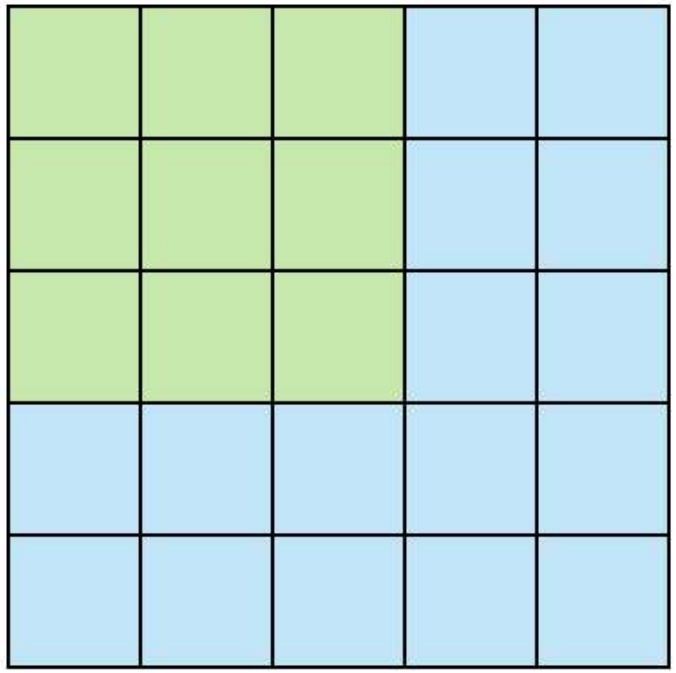
Step-3: Stride and Padding:

Stride specifies how much we move the convolution filter at each step. By default the value is 1, as you can see in the figure below.

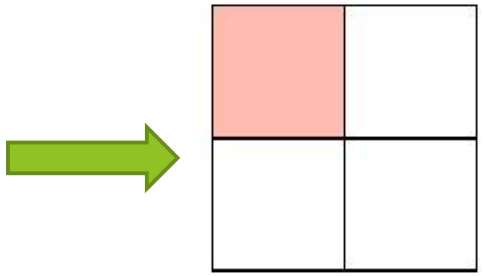


Stride 1

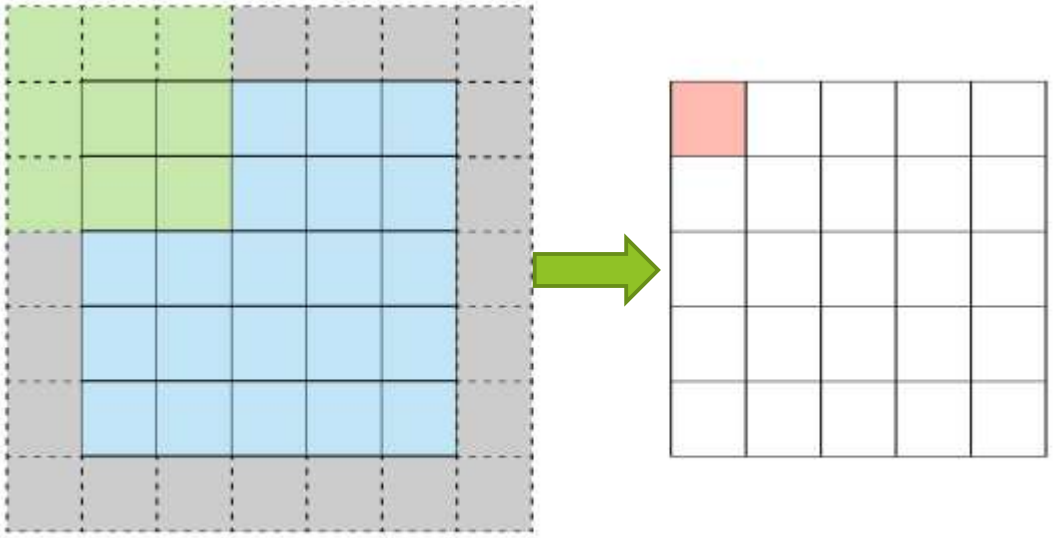
Feature Map



Stride 2



Feature Map



Stride 1 with Padding

Feature Map

We see that the size of the feature map is smaller than the input, because the convolution filter needs to be contained in the input. If we want to maintain the same dimensionality, we can use *padding* to surround the input with zeros. Check the animation below.

The gray area around the input is the padding. We either pad with zeros or the values on the edge. Now the dimensionality of the feature map matches the input. Padding is commonly used in CNN to preserve the size of the feature maps, otherwise they would shrink at each layer, which is not desirable.

Padding

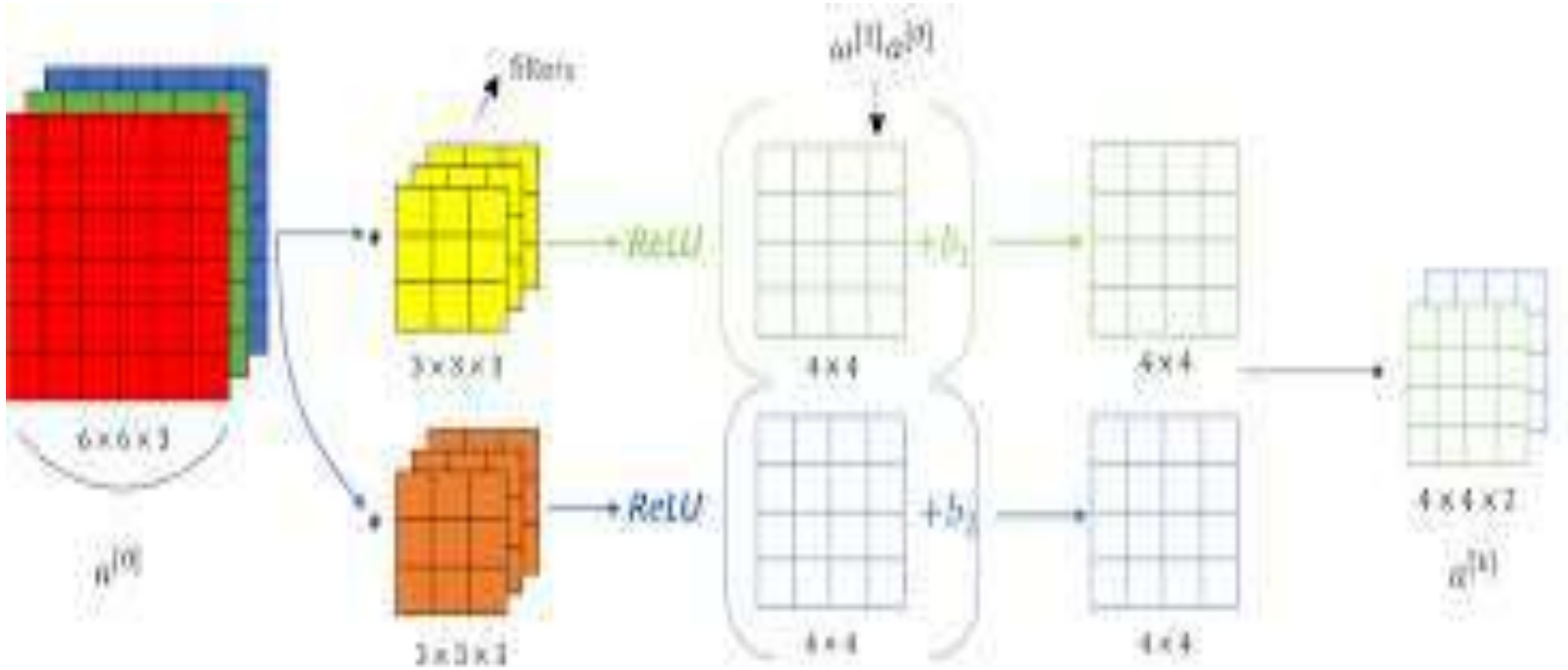
- Padding In order to to use deep neural networks we really need to use paddings.
- Assume that a 6x6 matrix convolved with 3x3 filter/kernel gives us a 4x4 matrix. To give it a general rule, if a matrix nxn is convolved with fxf filter/kernel give us n-f+1,n-f+1 matrix.
- The convolution operation shrinks the matrix if f>1. We want to apply convolution operation multiple times, but if the image shrinks we will lose a lot of data on this process.
- Also the edges pixels are used less than other pixels in an image.
- So the problems with convolutions are:
- Shrinks output. throwing away a lot of information that are in the edges.
- To solve these problems we can pad the input image before convolution by adding some rows and columns to it.
- We will call the padding amount P the number of row/columns that we will insert in top, bottom, left and right of the image.
- In almost all the cases the padding values are zeros.
- The general rule now, if a matrix nxn is convolved with fxf filter/kernel and padding p give us n+2p-f+1,n+2p-f+1 matrix.
- If n = 6, f = 3, and p = 1 Then the output image will have n+2p-f+1 = 6+2-3+1 = 6 . We maintain the size of the image.

Stride

- Strided convolution Strided convolution is another piece that are used in CNNs.
- We will call stride S .
- When we are making the convolution operation we used S to tell us the number of pixels we will jump when we are convolving filter/kernel.
- The last examples we described S was 1.
- Now the general rule are:
  - if a matrix nxn is convolved with fxf filter/kernel and padding p and stride s

    it give us (n+2p-f)/s + 1,(n+2p-f)/s + 1 matrix.

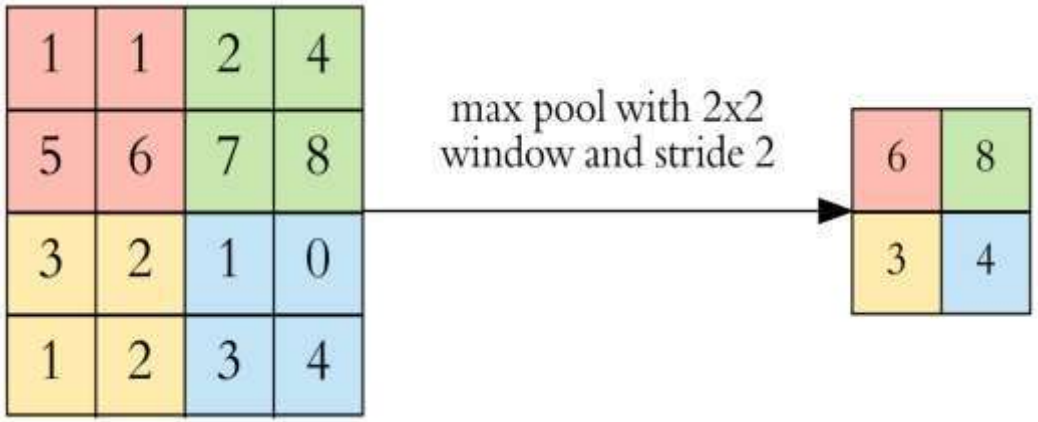
1Convolution layer



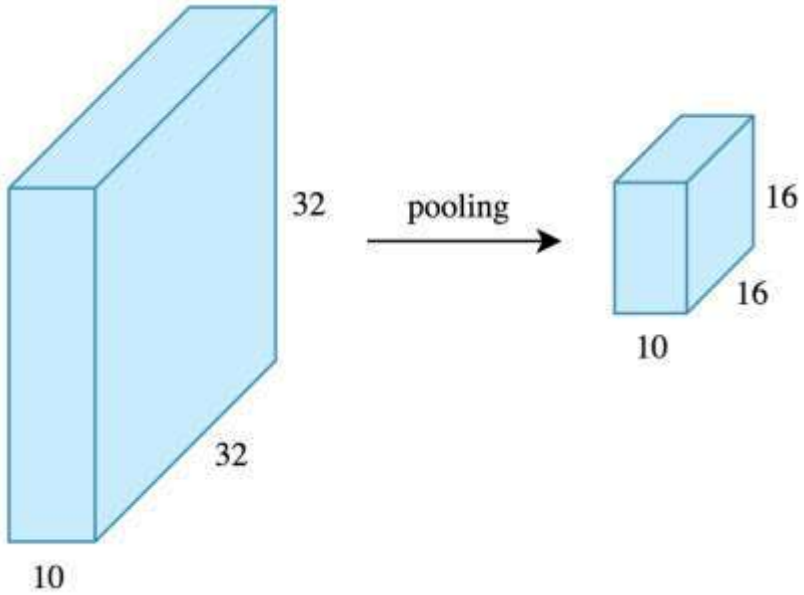
Step-4: Pooling

After a convolution operation we usually perform *pooling* to reduce the dimensionality. This enables us to reduce the number of parameters, which both shortens the training time and combats overfitting. Pooling layers downsample each feature map independently, reducing the height and width, keeping the depth intact

The most common type of pooling is *max pooling* which just takes the max value in the pooling window. Contrary to the convolution operation, pooling has no parameters. It slides a window over its input, and simply takes the max value in the window. Similar to a convolution, we specify the window size and stride.



- Here is the result of max pooling using a 2x2 window and stride 2. Each color denotes a different window. Since both the window size and stride are 2, the windows are not overlapping.
- Note that this window and stride configuration halves the size of the feature map. This is the main use case of pooling, downsampling the feature map while keeping the important information.



Now let’s work out the feature map dimensions before and after pooling. If the input to the pooling layer has the dimensionality 32x32x10, using the same pooling parameters described above, the result will be a 16x16x10 feature map. Both the height and width of the feature map are halved, but the depth doesn’t change because pooling works independently on each depth slice the input.

By halving the height and the width, we reduced the number of weights to 1/4 of the input. Considering that we typically deal with millions of weights in CNN architectures, this reduction is a pretty big deal.

Hyperparameters:

Let's now only consider a convolution layer ignoring pooling, and go over the hyperparameter choices we need to make. We have 4 important hyperparameters to decide on:

- **Filter size:**

  we typically use 3x3 filters, but 5x5 or 7x7 are also used depending on the application. There are also 1x1 filters which we will explore in another article, at first sight it might look strange but they have interesting applications. Remember that these filters are 3D and have a depth dimension as well, but since the depth of a filter at a given layer is equal to the depth of its input, we omit that.

- **Filter count:**

  this is the most variable parameter, it's a power of two anywhere between 32 and 1024. Using more filters results in a more powerful model, but we risk overfitting due to increased parameter count. Usually we start with a small number of filters at the initial layers, and progressively increase the count as we go deeper into the network.

- **Stride:**

  we keep it at the default value 1.

- **Padding:**

  we usually use padding.

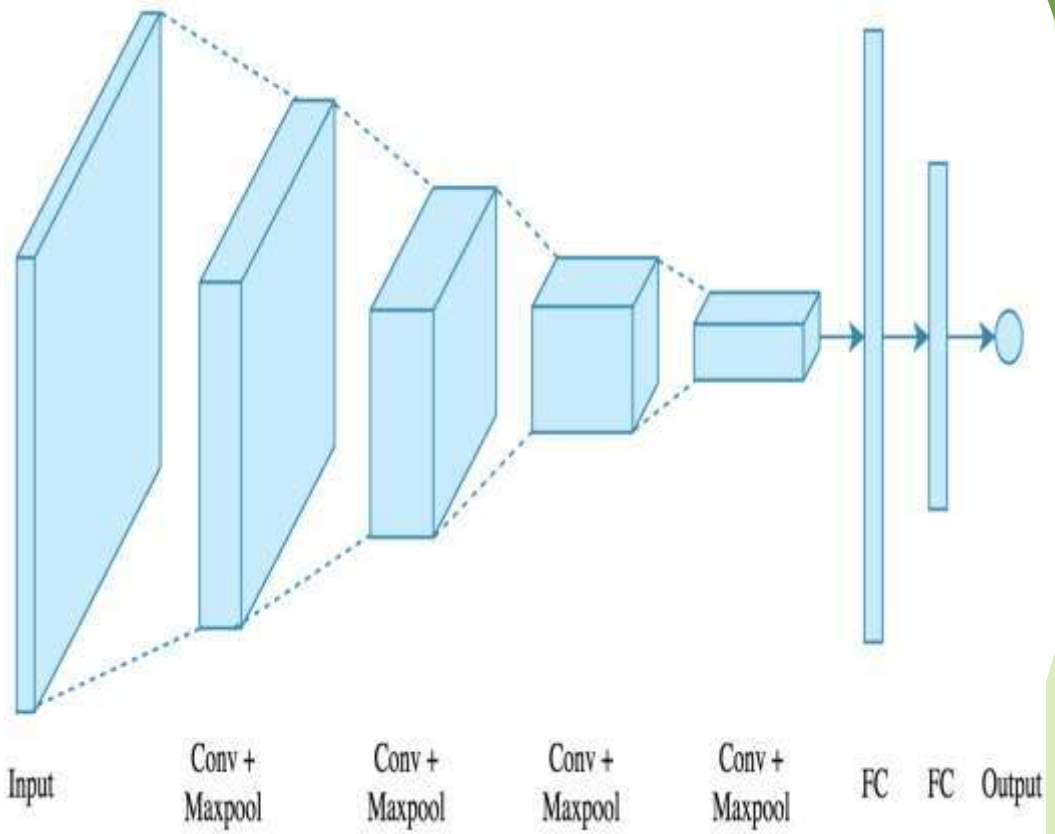
Step-5: Fully Connected

After the convolution + pooling layers we add a couple of fully connected layers to wrap up the CNN architecture. This is the same fully connected ANN architecture

Remember that the output of both convolution and pooling layers are 3D volumes, but a fully connected layer expects a 1D vector of numbers. So we *flatten* the output of the final pooling layer to a vector and that becomes the input to the fully connected layer. Flattening is simply arranging the 3D volume of numbers into a 1D vector, nothing fancy happens here.

Intuition:

- A CNN model can be thought as a combination of two components: feature extraction part and the classification part. The convolution + pooling layers perform feature extraction. For example given an image, the convolution layer detects features such as two eyes, long ears, four legs, a short tail and so on. The fully connected layers then act as a classifier on top of these features, and assign a probability for the input image being a dog.
- The convolution layers are the main powerhouse of a CNN model. Automatically detecting meaningful features given only an image and a label is not an easy task. The convolution layers learn such complex features by building on top of each other. The first layers detect edges, the next layers combine them to detect shapes, to following layers merge this information to infer that this is a nose. To be clear, the CNN doesn't know what a nose is. By seeing a lot of them in images, it learns to detect that as a feature. The fully connected layers learn how to use these features produced by convolutions in order to correctly classify the images.



Example:

We will use the following architecture: 4 convolution + pooling layers, followed by 2 fully connected layers.